

Спецификация и план разработки

Тема проекта: Визуализация алгоритма Флойда-Уоршелла

Язык реализации: Java 21 (OpenJDK)

Тип приложения: Desktopное с графическим интерфейсом.

Стек технологий: Java 21 (OpenJDK) + Swing (javax.swing) для интерфейса, отрисовка холста — через Graphics2D на JPanel (без сторонних UI-библиотек). Сборка — Maven, тестирование — JUnit 5, репозиторий — GitHub.

Обоснование выбора Swing вместо JavaFX: Swing входит в стандартную поставку JDK, что упрощает сборку исполнимого jar-файла без внешних зависимостей и модулей — JavaFX с версии 11 распространяется отдельно и требует либо подключения OpenJFX как библиотеки, либо сборки через jlink/jpackage, что при сжатых сроках практики создаёт дополнительный риск. Graphics2D в Swing полностью покрывает требования задания (отрисовка вершин, дуг, подсветка, анимация через Timer).

1. Общие положения

Проект представляет собой приложение с графическим интерфейсом для визуализации алгоритма нахождения кратчайших путей между всеми парами вершин во взвешенном ориентированном графе - алгоритма Флойда-Уоршелла.

Приложение позволит пользователю наглядно увидеть, как динамическое программирование последовательно улучшает оценки длин путей при добавлении промежуточных вершин.

2. Функциональные требования

Основные возможности пользователя

Управление графом:

Добавление вершин: Создание новой вершины на холсте по двойному щелчку мыши.

Редактирование ребер: Создание ориентированного ребра (дуги) между двумя вершинами через контекстное меню или панель инструментов. Задание веса ребра (целое или вещественное число). Вес может быть положительным, отрицательным или нулевым (наличие отрицательных циклов должно обрабатываться отдельно).

Удаление элементов: Удаление выбранной вершины или ребра (кнопка "Delete" или контекстное меню).

Перемещение вершин: Перетаскивание вершин по холсту для удобного расположения графа.

Ввод/Вывод данных:

Ручной ввод: Задание матрицы смежности графа в табличном виде прямо в интерфейсе приложения.

Загрузка из файла: Чтение графа из текстового файла (формат: количество вершин, координаты каждой вершины (X, Y), затем матрица смежности, где INF обозначает отсутствие ребра).

Сохранение в файл: Экспорт текущего состояния графа в тот же текстовый формат.

Управление визуализацией (Алгоритм):

Запуск: Кнопка "Запустить алгоритм".

Пошаговый режим: Кнопки "Шаг вперед" / "Шаг назад".

Индикация: Подсветка текущей обрабатываемой промежуточной вершины k на каждом шаге.

Обновление ребер: Визуальное изменение толщины/цвета ребер при улучшении кратчайшего пути (обновление матрицы $dist$).

Взаимодействие с результатом:

Матрица результатов: Отображение текущей и финальной матрицы кратчайших расстояний рядом с холстом.

Просмотр пути: Возможность кликнуть на ячейку финальной матрицы (между вершинами i и j) и увидеть на холсте путь (цепочку ребер), соответствующий этому кратчайшему пути (восстановление по матрице $next$).

3. Проектирование пользовательского интерфейса

Ниже представлен макет главного окна приложения.

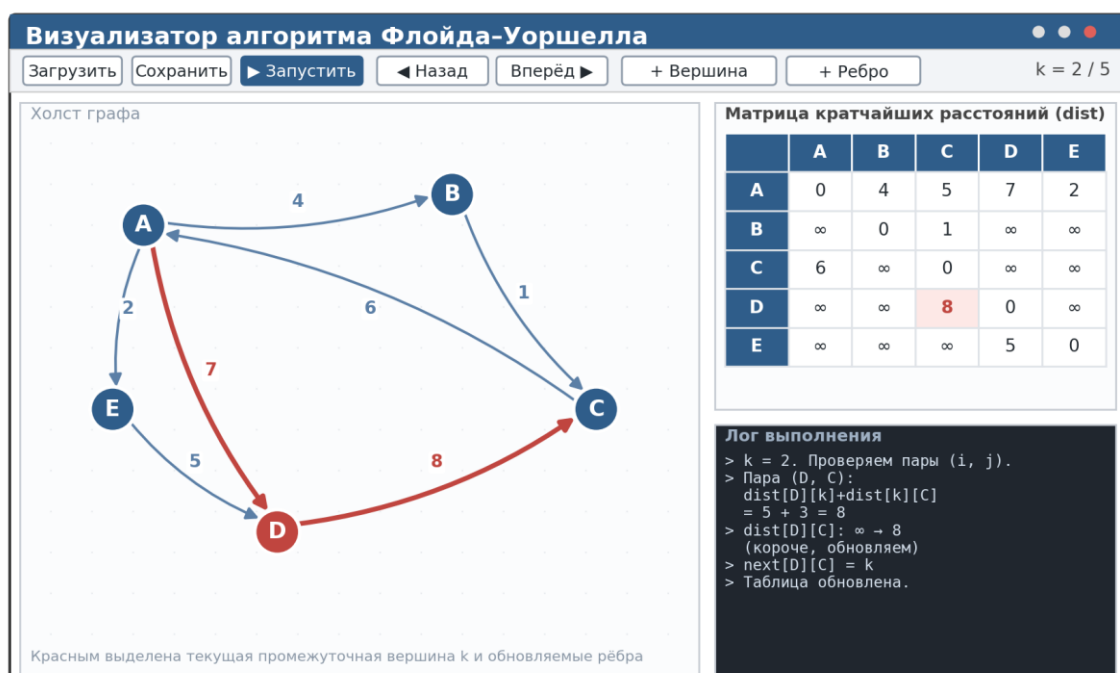


Рис. 1. Макет графического интерфейса.

Пояснение: Холст занимает левую часть окна. Правая панель разделена на область матрицы (таблица) и область лога/текстовых пояснений.

4. Процесс визуализации

При нажатии кнопки "Шаг" или автозапуске выполняются следующие действия (все конкретные значения и фразы написаны для примера):

Текущая промежуточная вершина k (внешний цикл) выделяется на холсте (например, обводится красным цветом).

В логе появляется сообщение: "Итерация $k = 2$. Проверяем все пары (i, j) ."

Визуально пробегаются пары (i, j) .

Если $D[i][k] + D[k][j] < D[i][j]$, то на холсте ребро (i, j) меняет цвет/толщину, и в матрице обновляется значение.

В логе: "Найден более короткий путь из A в C: $3 + 1 = 4 < 5$ ".

Обновление матрицы: Таблица справа перерисовывается с новыми значениями.

Окончание: Алгоритм завершается, когда $k == n$. Надпись:
"Алгоритм завершен. Найдены все кратчайшие пути."

5. План разработки (Итерации)

Итерация 0: Спецификация и план разработки, прототип (Дедлайн: 10 июля)

[Общее]: Создать репозиторий.

[Гриценко К.В. (Модель/Логика)]: Создать необходимые классы. Реализовать алгоритм Флойда-Уоршелла на массивах (без GUI, с вызовом из main для теста).

[Кузьмин Г.И. (Интерфейс/View)]: Разработать макет окна на Swing. Сделать холст (JPanel + Graphics2D) для отрисовки графа (круги, стрелки).

[Харченко Я.К. (Контроллер/Ввод)]: Реализовать загрузку графа из файла и парсинг в модель. Сделать кнопку "Загрузить".

Результат: Запускаемое окно с пустым холстом, кнопками и примером загруженного графа (статичного). Алгоритм работает, но результаты выводятся в консоль (или просто не шагово).

План тестирования (к сдаче прототипа, бонус до 1.5 очка): 1) unit-тесты алгоритма Флойда-Уоршелла (JUnit 5) — корректность на графах без циклов, обнаружение отрицательного цикла, граничный случай графа из одной вершины; 2) ручные сценарии проверки GUI — загрузка графа из файла, добавление/удаление вершины и ребра, пошаговое выполнение вперёд/назад, сверка итоговой матрицы dist с эталонным расчётом; 3) проверка сохранения графа в файл и повторной загрузки (round-trip).

Итерация 1: Базовая функциональность (Дедлайн: 13 июля)

[Гриценко К.В.]: Интегрировать алгоритм с визуализацией. Сделать метод step(), который выполняет одну итерацию k и возвращает список изменений (какие ребра обновились).

[Кузьмин Г.И.]: Реализовать динамическую отрисовку: отображение весов ребер, подсветку текущей вершины k, анимацию изменения цвета ребер при обновлении dist.

[Харченко Я.К.]: Реализовать кнопки "Шаг вперед", "Шаг назад" (стек состояний). Добавить таблицу для отображения матрицы dist.

Результат: Приложение позволяет загрузить граф, пошагово пройти алгоритм, видя изменения на холсте и в таблице.

Итерация 2: Улучшения (Дедлайн: 15 июля)

[Гриценко К.В.]: Реализовать восстановление пути по матрице next. Написать функцию, возвращающую список вершин для пути $i \rightarrow j$.

[Кузьмин Г.И.]: Реализовать "Drag-and-Drop" для вершин. Добавить контекстное меню (правый клик) для добавления/удаления ребер.

[Харченко Я.К.]: Связать клик по ячейке в матрице с отображением пути на холсте. Реализовать "Сохранение графа в файл".

Результат: Полноценное интерактивное приложение. Можно рисовать граф мышкой, запускать алгоритм и просматривать конкретные кратчайшие пути.

Итерация 3 (Финал): Полировка и Тесты (Дедлайн: 17 июля)

[Все]: Проверка на краевые случаи: граф с 1 вершиной, отрицательные веса (проверка на циклы).

[Гриценко К.В.]: Написать unit-тесты для алгоритма (JUnit 5): корректность на графах без циклов, с отрицательными весами без отрицательных циклов, определение отрицательного цикла, граф из 1 вершины.

[Кузьмин Г.И.]: Улучшить UI (иконки, тултипы, цветовая схема). Добавить полосу прогресса.

[Харченко Я.К.]: Собрать финальный .jar файл через Maven (запуск java -jar, совместимо с Java 21, без внешних зависимостей). Проверить запуск вне IDE на чистой машине. Написать разделы отчёта по своей части.

Результат: Готовый продукт, защищенный от ошибок, с красивым интерфейсом.

Роли:

Backend Developer (Гриценко К.В.):

Ответственность: Архитектура модели (Model), логика алгоритма, работа с файлами (ввод/вывод матриц).

UI/UX Developer (Кузьмин Г.И.):

Ответственность: Все, что касается графического интерфейса (Swing, Graphics2D), отрисовка графа, анимация, дизайн панелей.

Frontend/Controller Developer(Харченко Я.К.):

Ответственность: Связь между моделью и представлением (MVC), обработка событий мыши/клавиатуры, логика кнопок, управление состоянием (шаги), сборка проекта.

6. Журнал изменений требований